

# **11. Repaso de WebScraping y APIs**

**Ciencia de datos para la toma de decisiones II**

**Jorge**

**Juvenal**

**Campos Ferreira**

✉ [juvenal.campos@tec.mx](mailto:juvenal.campos@tec.mx)

## Propósito de la sesión

En esta sesión repasaremos términos y funciones útiles para trabajar con APIs y hacer webscraping:

1. Variables de ambiente
2. `lapply()`

## Variables de ambiente

Son pares clave-valor que el sistema operativo o que R utilizan para almacenar información de configuración.

Funcionan como un contexto que afecta a la ejecución de tu sesión en R, sin necesidad de estar definidas en el script.

# Variables de ambiente

Una variable normal existe solo dentro de tu entorno de trabajo.

**Una variable de ambiente vive en el sistema y es leída por R.** Puede afectar como funcionan los paquetes sin que tengas que ponerlas explícitamente en el código.

Suelen usarse típicamente para guardar llaves de acceso a APIs. Tiene la ventaja de la **seguridad** (evitas poner tus contraseñas en el código) y de **mantenibilidad** (cambias solo una contraseña en vez de cambiar la llave en cada código).

# Variables de ambiente

Ejemplo de cómo se vería el archivo .Renviron que carga las variables de ambiente.

Para modificar el archivo .Renviron utilizamos la función `usethis::edit_r_environ()`

```
# Ejemplo en un archivo .Renviron o en el sistema
R_LIBS_USER=/Users/juvenal/R/Library
DB_HOST=database.myserver.com
DB_USER=analyst_user
DB_PASS=SuperSecreta123
API_KEY_YOUTUBE=AIZAaSyAxxxxxxx-xxxxx-xxxx
API_KEY_SPOTIFY=Sp0tifyK3y_123456789
LANG=es_MX.UTF-8
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

# lapply()

Es una función que **nos permite hacer bucles sobre listas**. Tiene la ventaja que no nos exige generar tablas vacías al inicio de un bucle (como lo hacíamos con *for*). Nos devuelve como resultado también una lista.

```
lapply(X, FUN, ...)
```

- **X** : un objeto tipo lista o vector.
- **FUN** : la función que se aplicará a cada elemento.
- **...** : argumentos adicionales que quieras pasarle a la función.

# lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

**X**: puede ser 1:100 (del uno al 100), puede ser los nombres de los archivos, lo que sea sobre lo que querramos iterar.

**FUN**: La función que le queremos aplicar a los valores de **X**. Los argumentos adicionales se le pueden aplicar con ...

# lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

## Más sobre el FUN:

Al igual que en python, en un lapply podemos aplicar funciones anonimas (lambdas) que se van a ejecutar a todos los elementos de **X**.

# lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

# lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

# lapply()

Si la lista-resultado de un `lapply()` tiene las mismas dimensiones en todos sus elementos, con la función `do.call(rbind, .)` estas listas se pueden convertir en tablas.

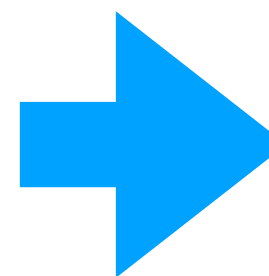
```
library(tibble)

# Tenemos un vector con nombres
personas <- c("Ana", "Luis", "Marta")

# Usamos lapply() para generar una lista de listas
mi_lista <- lapply(seq_along(personas), function(i) {
  tibble(
    nombre = personas[i],
    edad   = 20 + i * 5,
    ciudad = c("CDMX", "Monterrey", "Guadalajara")[i]
  )
})

# Convertimos la lista en tibble con do.call()
mi_tibble <- mi_lista %>% do.call(rbind, .)

mi_tibble
```



| nombre | edad  | ciudad      |
|--------|-------|-------------|
| <chr>  | <dbl> | <chr>       |
| Ana    | 25    | CDMX        |
| Luis   | 30    | Monterrey   |
| Marta  | 35    | Guadalajara |

## Actividad práctica

**Acceda al script “codigo\_apis\_sesion\_2.R” y ejecute el código. Introduzca sus claves privadas. Realice las actividades solicitadas.**

**En caso de que quede tiempo de la sesión, practique *web scraping*. Obtenga todas las versiones estenograficas de las conferencias de la presidenta que ha dado en 2025, disponibles en este enlace: <https://www.gob.mx/presidencia/es/archivo/articulos?idiom=es&order=DESC&page=2>**